

CyberShield AI

Month 2 Month 2 — Project Setup, Database & Authentication

Goal

The primary goal of the second month of the **CyberShield AI** project was to establish the complete development environment, design and implement the database architecture, and build a secure authentication system for the platform. This phase focused on preparing the technical foundation required for developing all future modules.

During this month, the project environment was configured using Python, Flask, and all required dependencies. The database models were created to store user information, security scans, threat intelligence, and training data. A complete authentication system with user registration, login, password management, session handling, and JWT-based API authentication was also implemented.

Completing this phase ensures that the project has a secure, scalable, and maintainable backend architecture, allowing future cyber security modules to be integrated efficiently.

1. Project Environment Setup

The development environment for CyberShield AI was successfully configured using Python virtual environments to isolate project dependencies and maintain consistency across different systems. All required Python packages were installed through the requirements.txt file, ensuring compatibility and simplifying project deployment. An environment configuration file (.env) was created to securely store sensitive information such as secret keys, JWT authentication keys, and application settings. Using environment variables improves application security by preventing confidential data from being hardcoded into the source code.

The following software packages were installed during this phase:

- Flask – Web application framework
- Flask-SQLAlchemy – Object Relational Mapping (ORM)
- Flask-Login – User session authentication
- Flask-JWT-Extended – JWT-based API authentication
- Flask-Mail – Email service integration
- Flask-CORS – Cross-Origin Resource Sharing
- Werkzeug – Password hashing utilities
- python-dotenv – Environment variable management

This setup provides a stable and secure development environment for the CyberShield AI platform.

2. Flask Application Factory

The CyberShield AI project follows the **Flask Application Factory Pattern**, which is considered a best practice for building scalable Flask applications.

The `app.py` file initializes the Flask application, loads configuration settings, registers all Blueprints, configures database extensions, initializes authentication services, creates database tables, and sets up logging.

The Application Factory Pattern improves code organization, simplifies testing, supports multiple environments, and allows future expansion without major code modifications.

3. Configuration System

A centralized configuration system was implemented using the `config.py` file. Three different configurations were created to support different stages of development.

Development Configuration

Uses SQLite as the database with debugging enabled for local development.

Production Configuration

Uses PostgreSQL as the database with debugging disabled to provide better security and performance in the production environment.

Testing Configuration

Uses an in-memory SQLite database for automated testing without affecting production data.

Separating configurations ensures flexibility, security, and easier project maintenance.

4. Database Extensions

Several Flask extensions were integrated into the project to provide essential backend functionality.

- **SQLAlchemy** manages all database operations using Python objects.
- **LoginManager** handles user authentication and session management.
- **JWTManager** provides secure API authentication using JSON Web Tokens.
- **Flask-Mail** enables email-related features.
- **Flask-CORS** allows secure communication between the frontend and backend.

A **Token Revocation System** was also implemented to invalidate JWT tokens after logout, preventing unauthorized API access.

5. Database Models

The database structure was designed using SQLAlchemy models to support all features of the CyberShield AI platform.

The major database models include:

- Role
- User
- RevokedToken
- ScanHistory
- AuditLog
- ThreatCategory
- Threat
- IOC (Indicators of Compromise)
- TrainingModule
- Quiz
- QuizResult
- Certificate
- DailyChallenge
- Leaderboard

Each model stores specific application data and maintains relationships with other models, ensuring efficient data management and future scalability.

6. Database Seeding

An automatic database seeding mechanism was implemented to populate the database during the first application startup.

The initial data includes:

- Three default user roles
- Six cyber security threat categories
- One administrator account
- Training modules
- Quiz questions
- Sample cyber threats
- Indicators of Compromise (IOCs)
- Daily security challenge

Automatic seeding reduces manual configuration and enables immediate testing of application features.

7. Authentication System

A complete authentication system was developed to provide secure user access.

The authentication module supports:

- User Registration
- User Login
- User Logout
- Forgot Password
- Profile Management
- Password Change
- JWT Authentication
- JWT Token Refresh
- JWT Logout

Passwords are securely stored using **Werkzeug password hashing**, preventing plaintext password storage.

The authentication system supports both web sessions and REST API authentication.

Every authentication event such as login, logout, registration, and password changes is recorded in the audit logs for security monitoring.

8. Dashboard Module

A dynamic dashboard was created to provide users with a complete overview of their cyber security activities.

The dashboard displays:

- Total security scans
- Threats detected
- Suspicious activities
- URLs scanned
- Emails analyzed
- Cyber Security Score
- Training Progress
- Weekly Scan Statistics
- Recent Scan History
- Latest Threat Intelligence
- Leaderboard Rankings

Interactive charts and statistical summaries provide users with real-time insights into their security performance.

9. Conclusion

The second month of the **CyberShield AI** project successfully established the technical foundation required for future development. The project environment was configured, the Flask application architecture was implemented, database models were designed, and a secure authentication system was developed.

The implementation of database seeding, JWT authentication, audit logging, and the interactive dashboard ensures that the platform is secure, scalable, and ready for integrating advanced cyber security modules.

In Month 3

the project will focus on developing the core cyber security functionalities, including the **Website/URL Scanner, Email Phishing Analyzer, Password Strength Analyzer, Malware File Scanner, and AI-powered security classifiers**. These modules will transform the platform from a foundational system into a practical cyber security awareness and threat detection solution suitable for educational and research purposes.